Практическое занятие №5: «Рефакторинг программного кода»

Цель: Создать новую ветку для реализации интерактивного режима консольного приложения, провести рефакторинг кода для улучшения структуры, слить изменения в основную ветку, получив полностью рабочую версию программы.

Задачи:

1. **Переключение на основную ветку и создание новой ветки:**

Переключитесь на основную ветку:

git checkout master

Синхронизируйте локальную master с удаленной:

git pull origin master

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git checkout master
Already on 'master'
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git pull origin master
From https://github.com/PdgrsK/TodoListApp
 * branch            master     -> FETCH_HEAD
Already up to date.
```

Создайте новую ветку для реализации интерактивного режима:

git checkout -b feature/interactive-mode

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git checkout -b feature/interactive-mode
Switched to a new branch 'feature/interactive-mode'
```

2. **Полная реализация интерактивного режима:**

Models/TodoItem.cs

Этот файл остается без изменений со времен второго практического занятия, так как модель задачи не требует доработки.

Managers/TodoListManager.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using TodoListApp.Models;

namespace TodoListApp.Managers
{
    Ссылок: 2 | user, 43 мин назад | Автор: 1, изменений: 3
    internal class TodoListManager
    {
        private List<TodoItem> _todoList = new List<TodoItem>();
        private int _nextId = 1;

        // Конструктор: инициализация тестовыми данными
        Ссылок: 1 | user, 6 дн. назад | Автор: 1, изменение: 1
        public TodoListManager()
        {
            // Тестовые данные для демонстрации
            _todoList.Add(new TodoItem(_nextId++, "Buy groceries"));
            _todoList.Add(new TodoItem(_nextId++, "Read a book"));
            _todoList.Add(new TodoItem(_nextId++, "Go for a walk"));
        }

        // Метод для получения списка задач (можно использовать для внешних операций)
        Ссылок: 0 | 0 изменений | 0 авторов, 0 изменений
        public List<TodoItem> GetTodoList()
        {
            return _todoList;
        }

        // Метод для добавления новой задачи
        // Возвращает true при успехе, false при неудаче
        Ссылок: 0 | user, 6 дн. назад | Автор: 1, изменение: 1
        public bool AddTask(string description)
        {
            if (string.IsNullOrWhiteSpace(description))
            {
                Console.WriteLine("Error: Task description cannot be empty.");
                return false;
            }
            _todoList.Add(new TodoItem(_nextId++, description));
            Console.WriteLine($"Success: Task '{description}' added.");
            return true;
        }

        // Метод для переключения статуса выполнения задачи
        // Возвращает true при успехе, false при неудаче (неверный ID)
        Ссылок: 0 | user, 42 мин назад | Автор: 1, изменение: 1
        public bool ToggleTaskCompletion(int taskId)
        {
            var taskToToggle = _todoList.FirstOrDefault(t => t.Id == taskId);
            if (taskToToggle == null)
            {
                Console.WriteLine($"Error: Task with ID {taskId} not found.");
                return false;
            }

            taskToToggle.IsCompleted = !taskToToggle.IsCompleted;
            Console.WriteLine($"Success: Task {taskId} status updated to {taskToToggle.GetStatusDisplay()}.");
            return true;
        }
```

```
        // Метод для отображения списка дел в консоли
        Ссылок: 1 | user, 6 дн. назад | Автор: 1, изменение: 1
        public void DisplayTodoList()
        {
            Console.Clear(); // Очищаем консоль для лучшей читаемости
            Console.WriteLine("--- Your To-Do List ---");
            if (_todoList.Count == 0)
            {
                Console.WriteLine("Your list is empty!");
            }
            else
            {
                foreach (var item in _todoList)
                {
                    Console.WriteLine($"{item.Id}. {item.GetStatusDisplay()} {item.Description}");
                }
            }
            Console.WriteLine("-----------------------");
        }



    }
}
```

Program.cs:

```
using System;
using TodoListApp.Models;
using TodoListApp.Managers;

namespace TodoListApp
{
    Ссылок: 0 | user, 35 мин назад | Автор: 1, изменений: 4
    class Program
    {
        Ссылок: 0 | user, 35 мин назад | Автор: 1, изменений: 4
        static void Main(string[] args)
        {
            Console.WriteLine("Your Ultimate To-Do List!");

            var todoManager = new TodoListManager();

            RunInteractive(todoManager); // Запускаем основной интерактивный режим

            Console.WriteLine("\nPress any key to exit...");
            Console.ReadKey();
        }

        // Метод, отвечающий за основной интерактивный цикл приложения
        Ссылок: 1 | 0 изменений | 0 авторов, 0 изменений
        static void RunInteractive(TodoListManager manager)
        {
            while (true)
            {
                manager.DisplayTodoList(); // Показываем текущий список дел
                Console.WriteLine("\nAvailable commands: add, toggle, exit");
                Console.Write("Enter command: ");
                string command = Console.ReadLine()?.Trim().ToLower(); // Используем ?. для null-conditional, если Console.ReadLine() вернет null

                bool operationSuccessful = false; // Флаг для индикации успеха операции

                switch (command)
                {
                    case "add":
                        Console.Write("Enter the description for the new task: ");
                        string description = Console.ReadLine();
                        operationSuccessful = manager.AddTask(description);
                        break;

                    case "toggle":
                        Console.Write("Enter the ID of the task to toggle completion: ");
                        string taskIdInput = Console.ReadLine();
                        if (int.TryParse(taskIdInput, out int taskId))
                        {
                            operationSuccessful = manager.ToggleTaskCompletion(taskId);
                        }
                        else
                        {
                            Console.WriteLine("Error: Invalid ID format. Please enter a number.");
                            operationSuccessful = false; // Считаем операцию неуспешной
                        }
                        break;

                    case "exit":
                        return; // Выход из цикла RunInteractive, а затем и из программы

                    default:
                        Console.WriteLine("Error: Unknown command. Please try again.");
                        operationSuccessful = false; // Неизвестная команда - неуспех
                        break;
                }
```

```
            // Небольшая пауза, чтобы пользователь увидел сообщение об успехе/ошибке
            if (operationSuccessful || command == "add" || command == "toggle" || command == "exit" || command == null) // Если команда была valid, и мы не выходим
            {
                Console.WriteLine("\nPress Enter to continue...");
                Console.ReadLine();
            }
            else
            {
                // Если была ошибка, но не exit, тоже можно дать паузу
                Console.WriteLine("\nPress Enter to continue...");
                Console.ReadLine();
            }
        }
    }
}
}
```

3. **Тестирование интерактивного режима:**

Запустите приложение (**dotnet run** или через Visual Studio).

Попробуйте все команды: add (с корректным и некорректным описанием), toggle (с корректным Id, несуществующим Id, нечисловым вводом), exit.

Убедитесь, что все работает как ожидалось.

```
C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp\bin\Debug\net8.0\T
--- Your To-Do List ---
1. [ ] Buy groceries
2. [ ] Read a book
3. [ ] Go for a walk
----------------------

Available commands: add, toggle, exit
Enter command: add
Enter the description for the new task: Pet the dog
Success: Task 'Pet the dog' added.

Press Enter to continue...
```

```
C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp\bin\Debug\net8.0\
--- Your To-Do List ---
1. [ ] Buy groceries
2. [ ] Read a book
3. [ ] Go for a walk
4. [ ] Pet the dog
----------------------

Available commands: add, toggle, exit
Enter command: toggle
Enter the ID of the task to toggle completion: 1
Success: Task 1 status updated to [x].

Press Enter to continue...
```

```
Консоль отладки Microsoft Visual Studio
--- Your To-Do List ---
1. [x] Buy groceries
2. [ ] Read a book
3. [ ] Go for a walk
4. [ ] Pet the dog
----------------------

Available commands: add, toggle, exit
Enter command: exit

Press any key to exit...

C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp\bin\Debug\net8.0\TodoListApp.exe (процесс 2708) завершил работу
 с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

4. **Фиксация изменений:**

Зафиксируйте все завершенные изменения:

cd TodoListApp

git add

git commit -m "Feat: Implement interactive mode with add, toggle, exit commands"

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git add .
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git commit -m "Feat: Implement interactive mode with add, toggle, exit commands"
[feature/interactive-mode dc15673] Feat: Implement interactive mode with add, toggle, exit commands
 2 files changed, 102 insertions(+), 44 deletions(-)
```

5. **Слияние ветки в master:**

Переключитесь на основную ветку:

git checkout master

Синхронизируйте master с удаленной:

git pull origin master

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git checkout master
Switched to branch 'master'
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git pull origin master
From https://github.com/PdgrsK/TodoListApp
 * branch            master     -> FETCH_HEAD
Already up to date.
```

Слейте вашу ветку feature/interactive-mode в master:

git merge feature/interactive-mode

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git merge feature/interactive-mode
Updating 16dd528..dc15673
Fast-forward
 TodoListApp/Managers/TodoListManager.cs | 78 ++++++++++++++++++---------------------
 TodoListApp/Program.cs                  | 68 ++++++++++++++++++++++++++++++++++----
 2 files changed, 102 insertions(+), 44 deletions(-)
```

Разрешение конфликтов (если возникнут): Если вы вносили изменения в Program.cs на основной ветке, которые конфликтуют с вашими изменениями в feature/interactive-mode, разрешите их так же, как на предыдущем занятии.

Загрузите результат слияния:

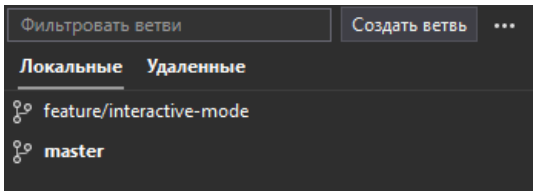
git push origin master

```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git push origin master
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 2.19 KiB | 561.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/PdgrsK/TodoListApp.git
   16dd528..dc15673  master -> master
```

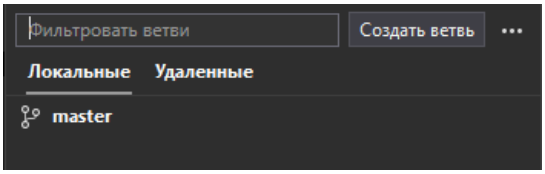
6. **Удаление ветки (опционально):**

Удалите локальную ветку:

git branch -d feature/interactive-mode




```
PS C:\Users\Подгурская\source\repos\TodoListApp\TodoListApp> git branch -d feature/interactive-mode
Deleted branch feature/interactive-mode (was dc15673).
```



Результат: Реализован интерактивный режим консольного приложения, включая добавление, отметку выполнения и выход. Код рефакторирован для лучшей структуры и удобства использования. Все изменения успешно слиты в основную ветку main, и конечная рабочая версия программы загружена на GitHub.

Структура проекта:

```
TodoListApp/
├── bin/
├── obj/
├── Managers/
│   └── TodoListManager.cs
├── Models/
│   └── TodoItem.cs
├── Program.cs
└── TodoListApp.csproj
```

Этот набор файлов представляет собой полностью рабочую версию вашего консольного приложения для управления списком дел с интерактивным режимом, готовым к дальнейшему развитию